CS 240 - Lab 3

TAs: Malay & Niral

Spring 2025

Welcome to the third graded lab of this course!!

The graded portion of this lab deals with two techniques that can be used for classification namely Least Squares Classification and Logistic Regression. Least squares classification is done using OLS and Logistic Regression will be done via gradient descent building naturally on the previous lab. It is followed by a deeper investigation into logistic regression and specifically on methods to deal with class inbalance. The outlab is a question is a music instrument classification problem that demonstrates the power of the techniques we have developed till now... Utilizing them to solve a somewhat non-trivial task.

Instructions:

- This lab has 3 questions. The first Question is the graded inlab. While the third question will be part of the outlab.
- Create a folder on your Desktop with the following directory structure. The folder must contain only the two specified files:

```
submission_<roll_number>/
|-- q1_solve.py
```

- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.

Q1. Gradient Descent for Classification

Consider a binary classification problem with input $X \in \mathbb{R}^d$ and class label $Y \in \{-1, +1\}$. Let $f(x)$ denote our prediction function of Y on X.

Optimal Classifier from the Regression Function

We begin by naively extending the framework of linear regression to classification.
Consider $f(x) = \mathbb{E}[Y \mid X = x]$ to be the regression function of Y on X, which minimises the squared error loss $\mathbb{E}\big[(Y - f(X))^2\big]$.
Since $Y \in \{-1, +1\}$, we have

$$\mathbb{E}[Y \mid X = x] = (+1)P(Y = +1 \mid X = x) + (-1)P(Y = -1 \mid X = x)$$

Thresholding the regression function at 0 yields the Bayes optimal classifier under zero-one loss:

$$\hat{y}(x) = \text{sign}(f(x)).$$

We model the regression function using a linear model $f_w(x) = w^\top x$, and use gradient-descent-based methods to estimate w by minimising the empirical MSE. Please note that optimization is to be done on the regression function, and then once fit, to threshold.
The resulting classifier is

$$\hat{y}(x) = \text{sign}(w^\top x).$$

This procedure is known as *least squares classification*. While simple and computationally convenient, it does not explicitly account for the discrete nature of the classification problem.

Task-1: Least Squares Classification via Gradient Descent

Use the provided template code to implement Least Squares Classification using Standard Gradient Descent Method. Note that using the closed form solution for OLS is not allowed.

There are several problems in this approach. Try to think of the same, and feel free to discuss after the graded lab ends. These limitations motivate alternative loss functions such as logistic and hinge losses, which are more naturally aligned with classification objectives.

Using Logistic Loss

We continue to use a linear prediction function $f_w(x) = w^\top x$, with labels $y \in \{-1, +1\}$.
The *logistic loss* is defined as

$$\ell_{\log}(w; x, y) = \log\big(1 + \exp(-y w^\top x)\big).$$

The empirical risk minimisation problem is

$$\min_{w \in \mathbb{R}^d} \frac{1}{N} \sum_{i=1}^{N} \log\big(1 + \exp(-y_i w^\top x_i)\big).$$

Unlike the squared error loss, this objective does not admit a closed-form solution. Consequently, we must rely on iterative optimisation algorithms such as gradient descent.

Task-2: Logistic Regression via Gradient Descent

Using the provided template code, implement the logistic loss and its gradient, and use the same to implement gradient descent method to minimize the empirical logistic loss. Also, track and plot the loss value as a function of iterations.
Use the same learning rate and stopping criterion as in Task 1, and compare the optimisation behaviour of least squares and logistic loss.

Also be careful while looking at dimensions in the update step (And throughout this lab for that matter). If the dimensions of the output do not match the function signature then you WILL NOT get credit for that function
And, finally in case you cannot get a function working fill out other functions assuming that the function is correct. You will still receive partial credit for correct function implementations.

Graded Lab Ends :)

Primer: Evaluation Metrics for Binary Classification

In binary classification, a model assigns each input x to one of two classes, commonly referred to as the *negative* class (0) and the *positive* class (1). In many real-world problems, different types of errors carry different costs, and the classes are imbalanced.
This primer introduces the most commonly used evaluation tools for such settings.

Confusion Matrix

The confusion matrix summarises the predictions of a classifier against the true labels:

	Predicted 0	Predicted 1
True 0	True Negative (TN)	False Positive (FP)
True 1	False Negative (FN)	True Positive (TP)

Each entry has a concrete interpretation:

- **TP**: correctly predicted positives
- **FN**: positives incorrectly predicted as negatives
- **FP**: negatives incorrectly predicted as positives
- **TN**: correctly predicted negatives

The confusion matrix is often more informative than accuracy, as it exposes which types of errors the model makes.

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

While simple, accuracy can be misleading when one class dominates the dataset. A classifier that always predicts the majority class may achieve high accuracy while completely failing to detect the minority class.

Recall and Precision

Two metrics that focus on the positive class are *recall* and *precision*.

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Recall measures the fraction of actual positive examples that are correctly identified. High recall means few false negatives.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Precision measures the fraction of predicted positives that are actually positive. High precision means few false positives.
There is often a trade-off between recall and precision: increasing one typically decreases the other.

F1 Score The F1 score combines precision and recall into a single metric:

$$\mathrm{F1} = 2 \cdot \frac{\mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}.$$

The F1 score is the harmonic mean of precision and recall and penalizes extreme imbalance between them. It is commonly used when both false positives and false negatives are important.

Probabilistic Outputs and Thresholding

Many classifiers, such as logistic regression, output a *score* or *probability* $\hat{p}(x) \in [0, 1]$ rather than a hard label.
A prediction is obtained by choosing a threshold τ:

$$\hat{y} = \begin{cases} 1, & \hat{p}(x) \geq \tau, \\ 0, & \hat{p}(x) < \tau. \end{cases}$$

Changing τ does not change the trained model, but it does change the confusion matrix and all derived metrics.

ROC Curve

The Receiver Operating Characteristic (ROC) curve visualizes model performance across all thresholds.
For a given threshold τ, define:

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP + FN}, \quad \text{False Positive Rate (FPR)} = \frac{FP}{FP + TN}.$$

The ROC curve plots TPR versus FPR as τ varies from 1 to 0.

Interpretation

- Each point corresponds to a different threshold.
- The diagonal line represents random guessing.
- Curves closer to the top-left corner indicate better performance.

Area Under the ROC Curve (AUC)

The Area Under the ROC Curve (AUC) is a scalar summary of the ROC curve: $\mathrm{AUC} \in [0, 1]$.

- An AUC of 0.5 corresponds to random performance,
- An AUC of 1.0 corresponds to perfect ranking.

AUC measures how well the model ranks positive examples above negative ones, but it does not specify performance at any particular threshold.

Q2. Classification Under Class Imbalance: Thyroid Data

In this problem, you will study binary classification under class imbalance using a real medical dataset: the *Thyroid Disease Data* from the UCI Machine Learning Repository.
The dataset contains clinical and laboratory information of patients evaluated for thyroid dysfunction. The task is to predict whether a patient belongs to the thyroid sick category or is negative (euthyroid). This is a naturally imbalanced binary classification problem, where the positive class (thyroid sick) is relatively rare but clinically important.
You will implement logistic regression using gradient descent, analyze its behavior using appropriate evaluation metrics, and experiment with strategies to address class imbalance.

Dataset Description

The dataset contains information on 3772 patients, with the following characteristics:

- Numerical features (e.g., hormone levels such as TSH, T3, TT4)
- Binary clinical indicators (e.g., whether a test was done, prior treatments, medical conditions)
- A binary target variable `class`, indicating whether the patient is thyroid sick or negative

The positive class (thyroid sick) constitutes a small fraction of the dataset, making accuracy alone an inadequate evaluation metric.

Task A: Preprocessing

1. Identify categorical features and encode them using indicator (one-hot) variables.
2. Remove attributes with excessive missingness, and optionally drop non-clinical features.
3. Impute missing values using an appropriate strategy (e.g., KNN imputation).
4. Standardise numerical features to have zero mean and unit variance. (Think on why standardisation is important when using gradient-based optimisation methods.)

Note that you should ideally do imputation and standardisation per training fold and not on the whole data at once.

Task B: Baseline Logistic Regression

Train a logistic regression classifier using gradient descent on the standard logistic loss.

1. Evaluate the model using stratified cross-validation (split data into k folds while ensuring each fold maintains the same percentage of samples for every target class as the original dataset).
2. Compute the cross-validated confusion matrix.

Report and interpret the confusion matrix. In particular:

- Which type of error is more frequent?
- Why can accuracy be misleading in this setting?

Task C: ROC Curve

Logistic regression outputs class probabilities. Fix the trained model and vary the decision threshold used to obtain predictions.

1. Observe how recall and false positive rate change as the threshold is varied.
2. Plot the ROC curve by varying the classification threshold.
3. Compute the Area Under the ROC Curve (AUC).

Answer the following questions:

- Why is the ROC curve less sensitive to class imbalance than accuracy?
- Does a high AUC guarantee good performance for the minority class?

In standard binary classification, the learning objective assumes balanced class representation. However, with imbalanced data where one class significantly outnumbers the other (as in our dataset with $\approx 6\%$ positive cases), several issues arise:

- **Loss Function Dominance**: The majority class dominates the aggregate loss, allowing the model to achieve high accuracy by simply predicting the majority class.
- **Decision Boundary Bias**: Gradient-based optimization naturally biases the decision boundary toward the majority class.
- **Evaluation Metric Misleading**: Standard accuracy becomes a poor metric, as a naive "always-negative" classifier would achieve 94% accuracy while being clinically useless.

We categorize solutions into three philosophical approaches:

1. **Decision-Level**: Modify the prediction rule after training
2. **Algorithm-Level**: Modify the learning algorithm itself
3. **Data-Level**: Modify the training data distribution

Task D: Threshold Adjustment

The classifier outputs probabilities $P(y = 1|x)$. The default threshold is 0.5, but it can be tuned:

$$\hat{y} = \mathbb{I}[P(y = 1|x) \geq \tau]$$

Threshold Selection Methods

- **Cost-Minimizing**: $\tau^* = \arg\min_\tau [C_{FN} \cdot FN(\tau) + C_{FP} \cdot FP(\tau)]$
- **Fixed Sensitivity**: Choose τ to achieve specific recall (e.g., 0.95 for medical screening)
- **Youden's Index**: $\tau^* = \arg\max_\tau [\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1]$

Play around with the threshold and identify an appropriate threshold τ with a better performance on recall relative to the default threshold $\tau = 0.5$.

Task E: Class-Weighted Loss

The most direct algorithm-level approach modifies the loss function to assign different importance to examples from different classes, thereby indirectly penalising different types of errors.
For logistic regression with binary classification, standard cross-entropy loss for a single sample is:

$$\mathcal{L}(y_i, \hat{y}_i) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The weighted version incorporates class-specific weights:

$$\mathcal{L}_w(y_i, \hat{y}_i) = -[w_1 y_i \log(\hat{y}_i) + w_0(1 - y_i) \log(1 - \hat{y}_i)]$$

where w_1 and w_0 are weights for the positive (minority) and negative (majority) classes respectively.

Weight Selection Methods:

- **Inverse Frequency Weighting**:

$$w_1 = \frac{N}{2 \times N_1}, \quad w_0 = \frac{N}{2 \times N_0}$$

  where N is total samples, N_1 positive samples, N_0 negative samples.

- **Cost-Sensitive Learning**: Set weights based on misclassification costs:

$$w_1 = \text{Cost}(FN), \quad w_0 = \text{Cost}(FP)$$

  where FN is false negative and FP is false positive.

Experiment with these weights, and retrain the classifier:

- Compute the cross-validated confusion matrix.
- Compare recall and false positive rate with the baseline model.

Task F: Data-Level Strategies

Random Undersampling

Randomly remove majority-class samples until classes are balanced:

$$D_{\text{undersampled}} = \{(x_i, y_i) : y_i = 1\} \cup S_{\text{majority}}$$

where S_{majority} is a random subset of majority samples with $|S_{\text{majority}}| = |\{(x_i, y_i) : y_i = 1\}|$.
Note that to improve this method, instead of random undersampling, you can use sophisticated techniques to minimise information loss; for example, using clustering or distance based methods.

Random Oversampling

Randomly duplicate minority-class samples:

$$D_{\text{oversampled}} = \{(x_i, y_i) : y_i = 0\} \cup S_{\text{minority}}$$

where S_{minority} contains randomly selected (with replacement) minority samples s.t. $|S_{\text{minority}}| = |\{(x_i, y_i) : y_i = 0\}|$.

SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE assumes the feature space between minority samples is valid for interpolation. The number of synthetic samples generated is controlled by the oversampling ratio.
For each minority sample x_i:

1. Find its k-nearest neighbors in the minority class (typically $k = 5$)
2. Randomly select one neighbor x_j
3. Generate synthetic sample:

$$x_{\text{new}} = x_i + \lambda \times (x_j - x_i)$$

   where $\lambda \sim \text{Uniform}(0, 1)$

For each data-level strategy described above, modify only the training data (not the test data).

1. Retrain logistic regression using the standard logistic loss.

2. Compute and report the cross-validated confusion matrix.

Compare all approaches studied in this problem and answer the following:

- Which method improved minority-class recall the most?
- Which method led to the largest increase in false positives?
- Which approach would you prefer in a medical screening setting, and why?